

Agents, Workflows & RAG

Friday, 26 September 2025 6:29 PM

Agent

• Design

- Agent Types
 - ReAct
 - Chain Of Thoughts
 - Program Aided
- Agent Pattern
 - Plan and Execute
 - Network (Swarm and CrewAI),
 - Drawbacks :- Unreliable
 - Supervisor Agents / Master Slave Architecture

• Frameworks

- LangGraph(LangChain) :- Good to maintain state across multiple Agents
- Microsoft AutoGen / Crew AI / Swarm :- Good for Independent multi Agent
- HayStack :- RAG/Search Centric Agents

• Model Metrics

- Text to Text
 - GPT
 - Gemini
 - Llama
 - Mistral
 - Claude
 - GPT-4
- Vision model (Image to Text)
 - Llama-4
 - Scout
 - Maverick
 - LLaVA
 - PaliGemma :- Google OpenSource vLM model
- Text to Image
 - MidJourney
 - DALL-E
 - ImageGen
 - Muse
 - Stable Diffusion
- Text to Audio
 - MusicLM
 - MusicGen
 - Stable Audio
 - Suno
- Text to Video
 - Stable Video Diffusion

• Encoders

- LLM
 - text-embedding-3-large
- vLLM
 - OpenCLIP from OpenAI
 - SigCLIP (From Google)

• Best Practices

- Tooling
 - Input/Output
- State
- Memory
- Controls
- Scalability
- Prompting
- Tenancy
- Security
- Observability
- Governance

RAG

• Pipeline & Stages

- Ingestion and Pre-Processing
 - Parsing :- Extract raw text + metadata from source formats. Using tools like pdfminer, Apache Tika
 - Cleaning :- Remove boilerplate (headers/footers, navigation links).
 - Normalization :- Lowercasing, Unicode normalization(unicodedata), lemmatization (spaCy).
- Chunking
 - Token-based :- Split by max tokens (e.g., 512–1000), aligns with embedding model's token limits but may cut mid-sentence.
 - sentence-based :- Natural boundaries, preserves meaning but may overshoot token limits.
 - semantic splitting - Split at topic boundaries , LangChain's RecursiveCharacterSplitter, semantic-text-splitter
 - Recursive - start by splitting paragraphs, then sentences, and finally by punctuation
- Tokenization & Embeddings
 - Dense
 - ◆ OpenAI :- text-embedding-3-large/small
 - ◆ BAAI :- bge-large-en
 - ◆ Sentence-Transformers := all-MiniLM-L6-v2
 - Sparse
 - ◆ BM25
 - ◆ ColBERT
 - Multi Model Embedding
 - ◆ Cohere Embed-4, OpenAI text embedding 3 large, GME 7 BN
- Indexing
 - HNSW (Hierarchical Navigable Small World) :- Builds a **multi-layer small-world graph**. It made up of two data structures i.e. Graph and skipped linked list
 - IVF :- Inverted File Index :- Trains **k-means** centroids At query time, find the **closest centroids** (nprobe)
- Retrieval
 - Query Embedding
 - Sparse Retrieval
 - Dense Retrieval
- Fusion
 - Reciprocal Rank Fusion (RRF)
 - Cross encoder
- Ranking
 - Top K Candidate

• Challenges

- Context window fitting
- OOV handling
- Token limits & Out of memory errors
- Retrieval relevance,
 - Hallucination
 - Lost in the middle
- Low Recall
- hybrid search latency
- Freshness
- prompt token limits

• Optimizations

- Simple Optimizations
 - Better Parsers
 - Chunk Sizes
 - Hybrid Search
 - BM-25 & Ensembles
 - MMR
 - Reciprocal Ranking
 - Metadata Filtering
 - Meta Data Filters
- Advanced Optimization
 - Query Expansion (Multi Query Retrieval) : User query → multiple paraphrases/sub-queries.
 - **LangChain MultiQueryRetriever**.
 - **Pyserini RM3 (PRF)**.
 - **ColBERT with PRF**.
 - Reranking
 - BM25/TF-IDF: Keyword-based filtering
 - ◆ **Pyserini + PyTorch** (BM25 + rerank).
 - Cross Encoders .
 - ◆ **HuggingFace Sentence Transformers** (cross-encoders).
 - LLM ReRanker
 - ◆ **LangChain Rerankers** (integrated).
 - Recursive retrieval :- Instead of one-shot retrieval, break queries into **sub-queries** iteratively and then merge

- Recursive retrieval :- Instead of one shot retrieval, break queries into **sub-queries** iteratively and then merge the results.
 - LangChain RecursiveRetriever.
 - LlamaIndex AutoMergingRetriever (progressive retrieval).
- Embedded tables :- Good for structured sources like tables and csv. Convert **rows/cells into embeddings**
 - LlamaIndex Structured Retriever.
 - OpenAI Embeddings + SQL retrievers
- Embedding fine tuning
- Multi-Document Agent
- Quantization
- Sharding & Routing

• Observability

- Retrieval evaluation :- How well system retrieves relevant information
 - Precision :- Of what you **retrieved**, how much was actually relevant?. $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
 - Recall :- Of what was **relevant in the corpus**, how much did you retrieve?. $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
 - Hit Rate :- least one relevant document is present in the top-k in One query
 - MRR :- Reciprocal of rank of the *first* relevant doc. if relevant doc is ranked 3rd $\rightarrow 1/3 = 0.33$.
 - NDCG(Normalized Discounted cumulative Gain) :- Measure the effectiveness of top ranked document .
 - MAP :- Mean average Precision , its average of all Precision of docs user watched
- Generator evaluation
 - BLUE(Precision)/ROUGE(Recall)
 - Grounded-ness/Faithfulness :- *Did the answer's claims come from the provided context?* (no hallucinations)
 - Context Relevance :- Did retrieval fetch the right evidence for this query??
 - Answer Correctness:- Answer compared with Ground Truth?
- Techniques
 - LLM as a Judge
 - Topic clustering
- Tools and Frameworks
 - RAGAS
 - LangSmith
 - Arize Phoenix
- Drift detection & handling the issues
 - Alerts
 - Actions

Security

• GuardRail

- Prompt injection Defense
 - Output filters, prompt guards
- Toxic Outputs
 - Output guardrails,
 - classifier-based filtering
- Privacy Risks
 - PII redaction
 - encrypted inference,
 - RBAC
- Tools & Frameworks
 - NVIDIA NEMO

• Developer Notes

- Knowledge Graph
 - Graph Database
 - Cypher Query Language
 - Integration with LangChain
- Cross-encoder
 - Precise contextual matching
- Prompts
 - Tuning
 - PII Reduction
 - injection defenses
- Workflow Frameworks
 - Langchain/LangGraph
 - CrewAI
 - AutoGen
 - ReAct
 - LaamaIndex
- Frameworks and Tools
 - Vector Databases (Compare diff databases)
 - Chroma DB
 - OpenSearch
 - PineCone (SAAS)
 - Milvus
 - In Memory Vector

- FAISS
- Tools
 - LlamaIndex, LangChain
 - LangExtract
 - LangSmith

MCP

Reinforcement Learning :- [\[Full Workshop\] Reinforcement Learning, Kernels, Reasoning, Quantization & Agents — Daniel Han](#)

Best Practices

Start hybrid retrieval + re-rank; tune chunk sizes per domain (code ≠ legal ≠ product pages).

Show sources (top-k with scores) and fail loudly when no good evidence.

Add semantic cache ahead of retrieval for repeat Qs.

Keep playbooks of tools and constrain agent loops; log every tool call.

Continuous evaluation on real traffic samples; track groundedness over time.

Be ready to answer: How would you eliminate hallucinations in a retrieval workflow? How do you keep agents safe and cost-bounded?

Cross-Cutting: What Interviewers Love Hearing
Rules of thumb

TTFT target: <300–800 ms (depends on model & stack).

Throughput target: talk tokens/sec/GPU and how batching + spec decoding improves it.

Fine-tuning: prefer PEFT; use full FT only to shift base capabilities.

RAG: Hybrid + re-rank > vector-only for most enterprise text.

Cost controls: adapterization per tenant, semantic caching, prompt compression, tiered models (small for easy, big for hard), early-exit routing.

Safety & governance: red-team prompts, jailbreak tests, policy engines, audit trails; data residency & deletion workflows.

Observability: unify LLM traces (prompt, context, latency, cost, guardrail outcomes) with app logs; drive A/B experiments off this.

Rapid-Fire Drills (use these tonight)

Design a fine-tuning pipeline for a domain (e.g., legal Q&A) – call out: data cleaning, PEFT choice, evals, rollout.

Whiteboard serving for 10M daily calls – call out: gateway, scheduler, batching, KV, autoscaling, caches, SLOs.

RAG design – call out: ingestion, hybrid search, re-rank, citations, eval loop, guardrails.

Trade-offs lightning round: QLoRA vs full FT, vLLM vs TGI vs TensorRT-LLM, Pinecone vs Qdrant vs OpenSearch, ReAct vs ToT.

War story: a latency/cost incident you “fixed”—metrics, bottleneck, change, result.

If you want, I can turn this into a printable 2-page cheat sheet and a 20-question mock interview set. Also happy to run a quick practice round on any one section right now—pick your poison: Fine-tuning, Serving, or RAG/Agents.

b) Production Implementation

Ingestion: connectors → parsing → normalization → chunk/summarize → embeddings → index (ANN: HNSW/IVF/PQ) + metadata.

Stores: Qdrant/Weaviate/Pinecone/FAISS/OpenSearch; freshness & TTL, per-tenant namespaces, legal hold.

Orchestration: LangGraph (deterministic stateful DAGs), LangChain/LlamaIndex, or homegrown FSMs; idempotent steps & retries.

Rerankers: Cohere/Instructor/monoT5 etc. for precision@k.

Guardrails & safety: input sanitization, prompt-injection defenses, tool whitelist, output filters (PII, secrets, toxicity).

Eval loop: RAGAS/TruLens for groundedness, answer relevance, faithfulness; golden sets + synthetic test variants.